PROJEKAT

Embedded operativni sistemi

TEMA PROJEKTA:

Prepoznavanje objekata na slici

MENTOR PROJEKTA:

Boris Radovanović

PROJEKAT IZRADIO:

Nikola Kovačić EE 77/2020
Ognjen Višnjić EE 217/2020

U Novom Sadu, 12.9.2025.

Sadržaj

1. Uvod 3
2. Razvoj drajvera za karakter uređaje 3
   2.1 Struktura *file_operations my_fops* 3
   2.2 Funkcije *REC_init* i *REC_exit* 4
   2.3 Funkcije *REC_open i REC_close* 4
   2.4 Funkcije *REC_read* i *REC_write* 4
   2.5 Funkcije za obradu slike u drajveru 5
3. Razvoj aplikacije 5
   3.1 Struktura i funkcionalnosti aplikacije 6
4. Zaključak 7

1. Uvod

Linuks je jedan od najpoznatijih i najviše korišćenih računarskih operativnih sistema današnjice. Veliku popularnost su mu donele činjenice da je besplatan, ne poseduje ga nijedna kompanija, razvijen je u skladu sa principima slobodnog softvera, što znači da svako ko poseduje kod može da ga menja i distribuira, i konstantno je održavan i unapređivan od strane velikog broja programera širom sveta.

Drajveri imaju specijalnu ulogu u Linuksovom jezgru (kernelu). Oni predstavljaju softverski sloj koji leži između aplikacije i hardvera. Potpuno kriju detalje funkcionisanja hardvera, i enkapsuliraju ga sa dobro definisanim interfejsom. Drugim rečima, drajveri predstavljaju spregu između hardvera i softvera (aplikacije), i olakšavaju komunikaciju između njih. S obzirom da postoji veliki broj različitih vrsta hardverskih uređaja, broj drajvera je takođe veoma veliki. Tempo pojavljivanja novih hardverskih uređaja i zastarevanja postojećih garantuje nepresušnu potrebu za drajverima. Osobina modularnosti omogućava da Linuksovo jezgro na datom računarskom ili embeded sistemu poseduje isključivo drajvere koji su mu potrebni.

Danas, povezivanje hardverskih komponenti sa procesorom se uglavnom vrši pomoću magistrala koje imaju mogućnost automatskog detektovanja periferijskih uređaja. Kernel, prilikom podizanja sistema, komunicira sa uređajima na magistrali, uzima sve neophodne informacije i enumeriše ih.

U ovom projektu umesto platformskog drajvera, razvijen je drajver za karakter uređaje, kao i korisnička aplikacija, za sistem koji prepoznaje i obeležava objekte na slici na osnovu njihove boje. Glavni fokus projekta je bio da se razvije funkcionalan sistem koji detektuje objekte pomoću algoritma za obradu slike, a ne na složenoj integraciji sa realnim hardverom i stablom uređaja. Platformski drajveri zahtevaju detaljno opisivanje uređaja u *device tree*-u, rad sa fizičkim adresama i integraciju sa stvarnim FPGA IP jezgrima, što značajno povećava kompleksnost i vreme razvoja. Korišćenje karakter uređaja omogućava pojednostavljenu simulaciju rada hardverskog sistema i IP jezgra, kroz virtuelne */dev* fajlove. Na taj način je moguće razviti i testirati ceo softverski sloj, nezavisno od FPGA platforme.

2. Razvoj drajvera za karakter uređaje

Da bi korisnička aplikacija mogla da koristi ovaj sistem, realizovan je karakter drajver koji upravlja pristupom memorijskim baferima i pokretanjem obrade, i obezbeđuje sinhronizaciju između više procesa. Drajver se sastoji iz sledećih šest funkcija i strukture *file_operations* koja definiše interfejs drajvera prema kernelu i korisničkom prostoru.

2.1 Struktura *file_operations my_fops*

Struktura *file_operations* (slika 1) predstavlja vezu između sistemskih poziva iz korisničkog prostora i funkcija koje implementira drajver. Na taj način kernel zna kako da komunicira sa drajverom i koje sve operacije on podržava. Polje *.owner* označava vlasnika

drajvera i u većini slučajeva, kao i u ovom projektu, postavljeno je na predefinisani makro *THIS_MODULE*. Ostale funkcije iz strukture biće detaljno opisane u nastavku dokumenta.

```
struct file_operations my_fops =
{
        .owner = THIS_MODULE,
        .read = REC_read,
        .write = REC_write,
        .open = REC_open,
        .release = REC_close,
};
```

Slika 1 Struktura file_operation my_fops

2.2 Funkcije *REC_init* i *REC_exit*

Funkcija *REC_init* se poziva prilikom učitavanja modula u kernel pomoću komande *insmod*, dok se funkcija *REC_exit* poziva prilikom brisanja modula pomoću komande *rmmod*. Drugim rečima, to su prva i poslednja funkcija koje se pozivaju pri radu sa drajverom.

Uloge funkcije *REC_init* su sledeće:

- Dinamički da alocira prve slobodne upravljačke brojeve pomoću funkcije *alloc_chrdev_region*
- Da napravi *inode* fajl u */dev* direktorijumu za svaki hardverski resurs, kojih u ovom slučaju ima 5 (4 za bram i 1 ip), pomoću naredbe *device_create*.
- Da poveže kreirane *inode* fajlove sa funkcionalnostima koje treba da se izvrše prilikom pozivanja neke od funkcija za rad sa fajlovima. To se postiže pomoću *c_dev* strukture.
- Svi semafori za kontrolu pristupa memorijskim baferima i IP jezgru se inicijalizuju kako bi se obezbedila pravilna sinhronizacija između više procesa.
- Ukoliko neka od naredbi ne bude uspešna, sve već izvršene naredbe treba da se ponište.

Uloga funkcije *REC_exit* je suprotna od *REC_init* funkcije. Njena uloga je da oslobodi upravljačke brojeve i da obriše kreirane *inode* fajlove.

2.3 Funkcije *REC_open i REC_close*

Iako postoji mogućnost da date funkcije obavljaju određenu funkcionalnost, u ovom projektu ove funkcije *REC_open* i *REC_close* su korišćene isključivo radi ispisivanje informacija u kernel log prilikom otvaranja i zatvaranja fajlova uređaja, što olakšava proveru ispravnosti komunikacije između aplikacije i drajvera.

2.4 Funkcije *REC_read* i *REC_write*

Drajver omogućava upisivanje originalne RGB slike u memorijske bafer-e (BRAM), pomoću funkcije *REC_write*, i pokretanje hardverske obrade slike, nakon čega se rezultati (binarna maska i obrađena slika) čuvaju u posebnim baferima. Ulazna slika se upisuje u bafere *bram_r_unmarked, bram_g_unmarked* i *bram_b_unmarked*, gde se svaki piksel upisuje na poziciju koja odgovara njegovom indeksu u slici. Preko čvora */dev/rec_ip*, pokreće se

hardversko IP jezgro koje vrši obradu slike koja je ubačena iz *userspace*, rezultati obrade se smeštaju u bafer *bram_r_marked*, odakle korisnička aplikacija može da isčita masku.

Funkcija *REC_write* na početku zaključava semafore kako se ne bi moglo pristupiti memorijama i ip tokom izvršavanja. Nakon toga funkcija prima podatke o slici iz korisničke aplikacije pomoću funkcije *copy_from_user* pri čemu se svaka boja raspodeljuje i upisuje u svoj namenjeni bafer. U ip-u se čitaju ulazni parametri i pokreće se obrada slike. Dobijeni rezultati se smeštaju u bafer *bram_r_marked* čije vrednosti možemo isčitati kasnije preko aplikacije. Na kraju *REC_write* funkcije bitno je osloboditi semafore kako bi se omogućio pristup drugim procesima.

Funkcija *REC_read* omogućava čitanje podataka iz drajvera u korisnički prostor. Čitanje radimo iz bafera *bram_r_marked* i u njemu se nalazi konačna maska obrađene slike. Pre početka svakog čitanja, semafor (*down_interruptible*) zaključava pristup ovoj memoriji. Vrednosti piksela iz memorije (*bram_**) se formatiraju u tekstualni niz pomoću *scnprintf(),* a zatim se prenose u korisnički prostor funkcijom *copy_to_user().* Kako bi znali da se čitanje izvršilo nad celim fajlom, funkcija koristi brojače (*cntrun, cntgun, cntbun, cntr, cntg, cntb*) i flag-ove (*endReadru, endReadgu, endReadbu*) kako bi simulirala ponašanje kraja fajla (EOF). Kada se pročita cela memorija, funkcija vraća 0 i resetuje stanje za sledeći poziv. Na kraju procesa bitno je podići semafor kako bi se on oslobodio i omogućio pristup drugim procesima.

2.5 Funkcije za obradu slike u drajveru

Funkcije koje se koriste za obradu slike su takođe implementirane u drajveru s obzirom da trenutna verzija drajvera emulira rad hardvera, kako bi se testirao kompletan sistem. U stvarnoj implementaciji na FPGA platformi, ovaj algoritam obrade slike bio bi realizovan u IP jezgru, dok bi drajver imao ulogu samo u komunikaciji i prenosu podataka između hardvera i aplikacije.

Prvi korak obrade slike je *color_thresholding* gde svaki piksel upoređujemo sa određenim parom pragova za taj objekat i pravimo binarnu masku, ako taj piksel upada u opseg, on postaje beo u masci, a ako ne upada u opseg postaje crn. Sledeći korak je dilatacija gde profinjujemo masku tako što širimo da i susedni pikseli od posmatranog takođe postanu beli u masci. Treći korak je erozija gde će piksel u masci ostati beo samo ako su svi pikseli oko njega u matrici 11x11 isto beli.

Ove tri funkcije predstavljaju algoritam obrade slike pri čemu se dobija crno bela maska, koja se kasnije u aplikaciji upoređuje sa određenim parametrima kako bi se odredilo da li je objekat validan da se prepozna.

3. Razvoj aplikacije

Aplikacija je napravljena u C++-u i koristi *OpenCV* kako bi korisniku na ekranu prikazala rezultate obrade slike. Njena glavna uloga je da omogući jednostavnu komunikaciju sa sistemom i prikaže šta se dešava u pozadini na jasan način.

Program se povezuje sa kernel drajverom preko posebnih fajlova u */dev/* direktorijumu, kao što su */dev/bram_r_unmarked, /dev/bram_g_unmarked, /dev/bram_b_unmarked* i */dev/bram_r_marked*. Pored toga, koristi i */dev/rec_ip* čvor kako bi pokrenuo IP jezgro koje obrađuje slike.

Putem aplikacije korisnik učitava sliku iz fajla, upisuje učitanu sliku u bram *node-ove* u kernel prostoru, bira kog pokemona želi da pronađe na slici i zatim pokreće obradu slike pri čemu se dobija rezultat u vidu koordinata gde se taj objekat nalazi na slici i koliko ih je

prepoznato na istoj. Takođe je moguć vizuelni prikaz, kako bi korisniku dalo bolju reprezentaciju o pronađenom objektu.

3.1 Struktura i funkcionalnosti aplikacije

Aplikacija funkcioniše kroz tekstualni meni u terminalu, gde korisnik može da bira opcije od 1 do 8. Proces rada aplikacije je organizovan kao niz jasnih koraka:

```
1 - Load data
2 - Write into bram
3 - Select Pokemon
4 - Start IP
5 - Read results
6 - Finish
7 - Show processed picture
8 - Exit
Please select an action:
```

Slika 2 Tekstualni meni u terminalu

Pritiskom na opciju jedan slika se iz binarnog fajla učitava u korisnički prostor. Opcijom dva, *write into Bram*, pikseli ove učitane slike se prenose i smeštaju u virtuelne BRAM */dev/bram_r_unmarked*, */dev/bram_g_unmarked* i */dev/bram_b_unmarke*. memorije koje drajver emulira. Opcija tri nudi korisniku da izabere kog pokemona želi da algoritam detektuje, pa se sa opcijom četiri rad ovog algoritam i započinje. Nakon obrade slike, odabirom opcije pet, rezultati se upisuju iz */dev/bram_r_marked* čvora u promenljivu u aplikaciji i ispis tih rezultata se vidi u terminalu. Na osnovu ovih rezultata, za svaki objekat se proverava da li je zatvorena kontura kao i da li zadovoljava parametre kao što su površina, visina i širina objekta. Ako je sve validno objekat se zaokružuje na slici i moguć je njegov vizuelni prikaz odabirom opcije 7, a podaci o njegovim kooridnatama, kao i informacija koliko ih ukupno ima na slici, se ispisuje u terminalu kada se izabere opcija 6.

Izlaz iz aplikacije je realizovan odabirom opcije 8.

Slika 3 Rezultat obrade slike

4. Zaključak

U ovom projektu implementiran je ceo sistem za obradu slike i prepoznavanje objekata po boji. Kombinuje Linuks drajver kao i korisničku aplikaciju koja komunicira sa hardverskim modulom preko pomenutog drajvera i izvršava softverski deo celokupnog algoritma.

Drajver omogućava da se slika iz *user space*-a prebaci u kernel *space* u kom se dalje pokreće obrada slike preko tri funkcije koje sadrže sav potreban algoritam da se nad slikom izvrše pravilne operacije da bi se validan objekat prepoznao. U drajveru su implementirani semafori koji omogućuju pravilan i sinhronizovan pristup memorijama.

Korisnička aplikacija omogućava jednostavnu interakciju, gde korisnik kroz niz opcija u meniju može učitati sliku, izabrati parametre boje, pokrenuti obradu, preuzeti rezultate i vizuelno proveriti tačnost algoritma. Generisana maska i prepoznati objekti zaokružuju se i prikazuju se na slici, dok se rezultati (površina, širina i pozicija objekata) detaljno ispisuju u terminalu.

Iako je hardver u ovom projektu simuliran na nivou drajvera i virtuelnih */dev* uređaja, kompletna struktura sistema verno oponaša rad na FPGA platformi i omogućava unapređivanje projekta tako što se relativno lako može implementirati direktno na FPGA čipu uz odgovarajuće promene.